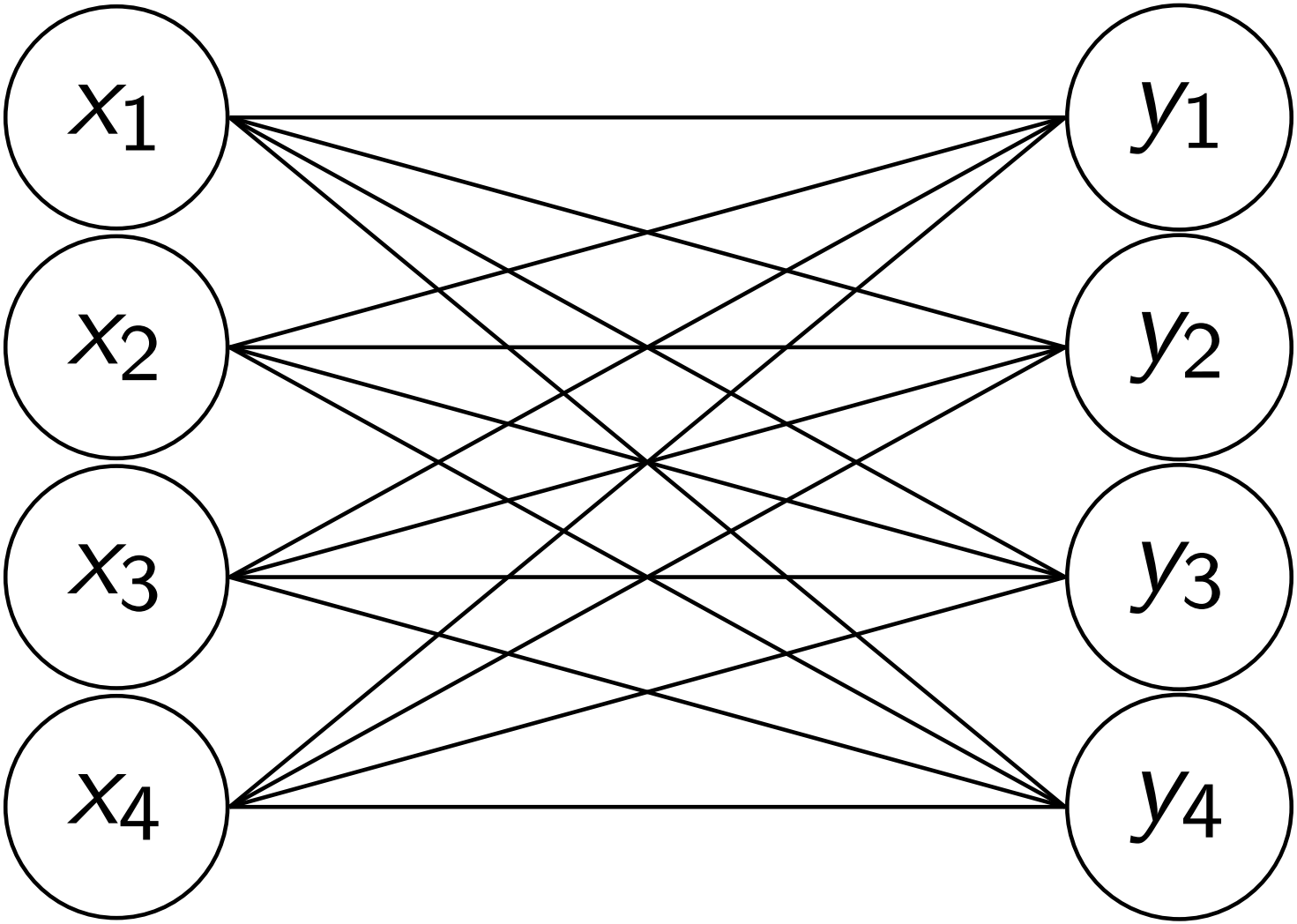
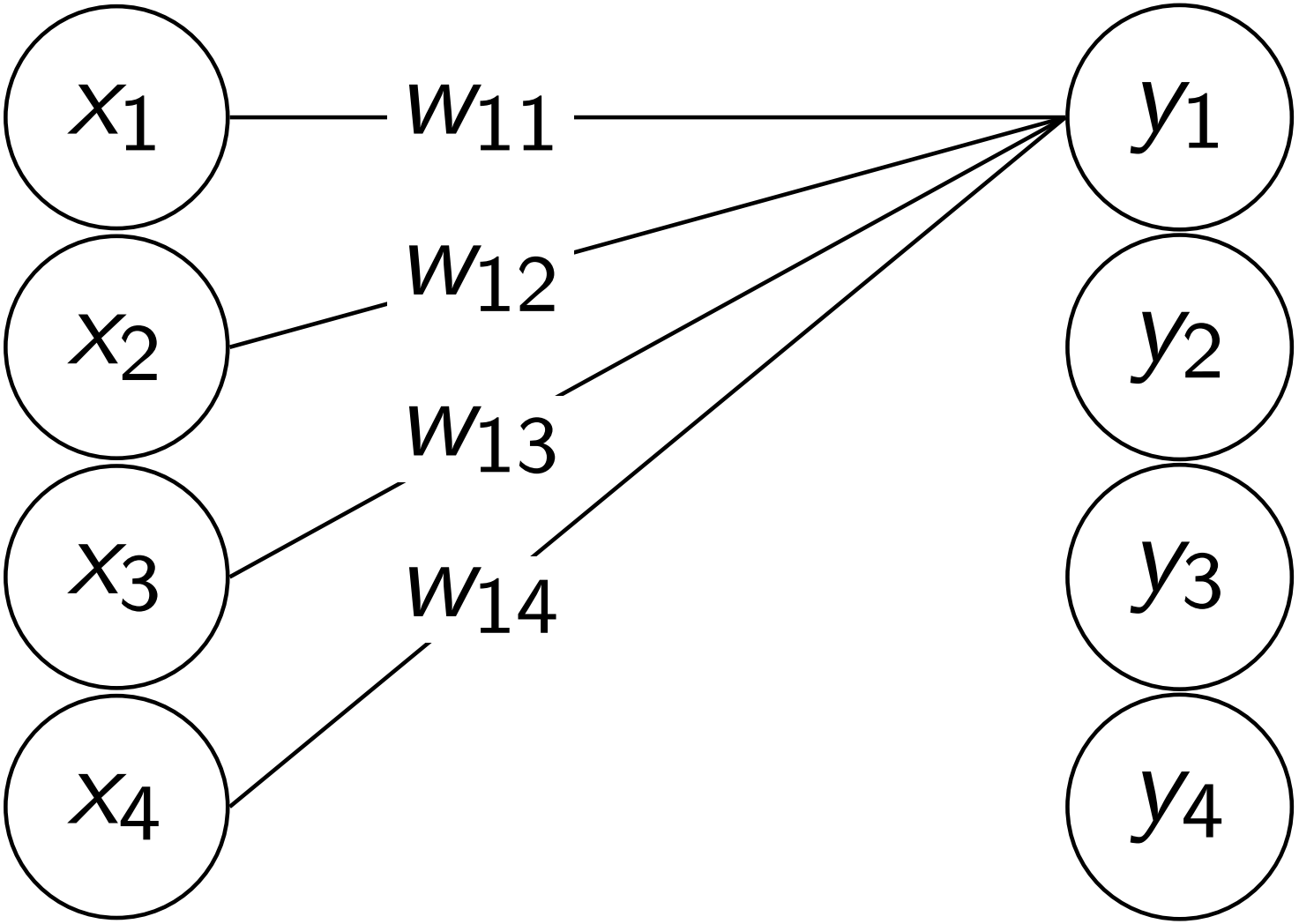
Table of Contents

1 Neural Networks and the Matrix Multiplication

Basic Neural Net (MLP/FFN)

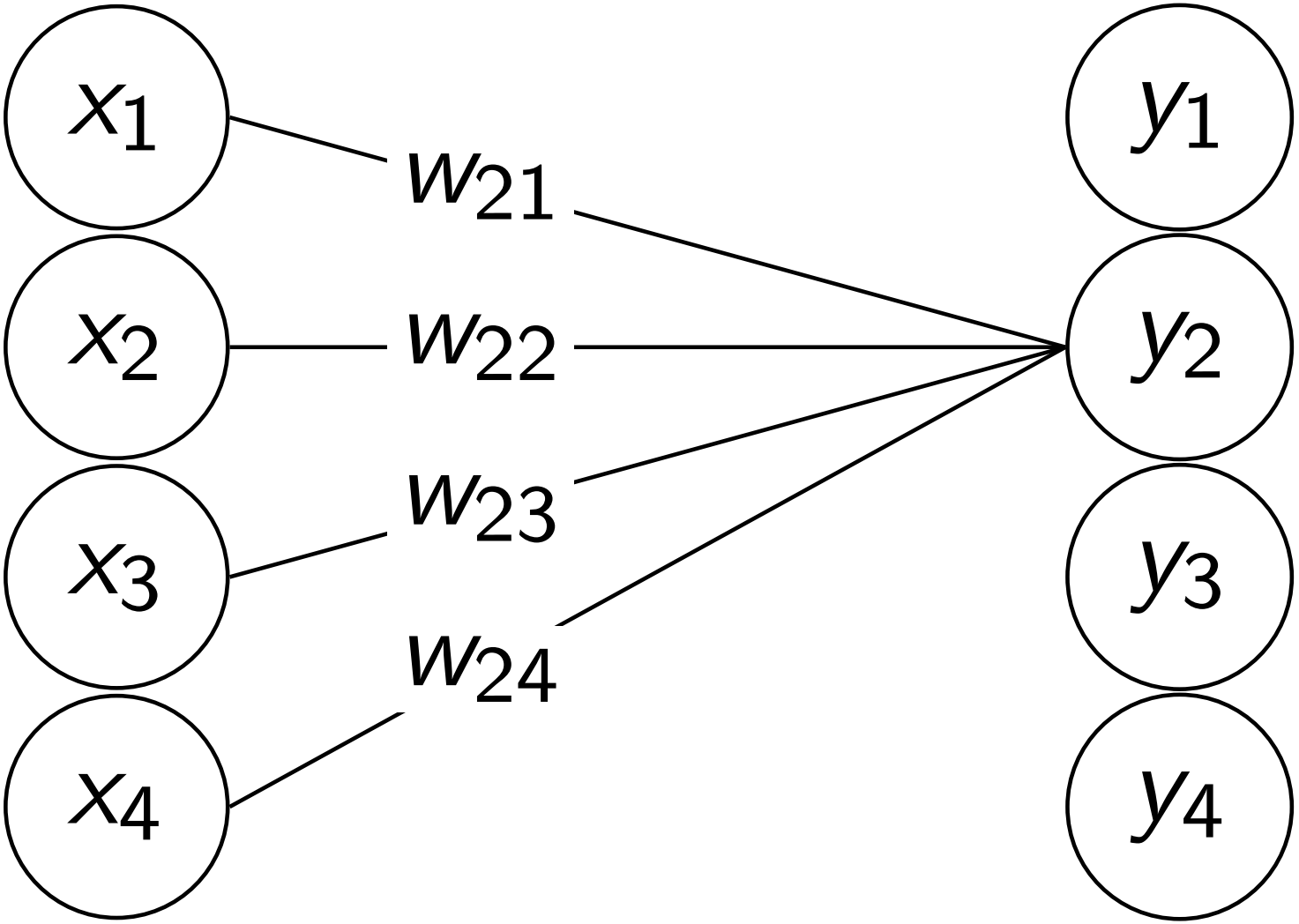


Basic Neural Net (MLP/FFN)



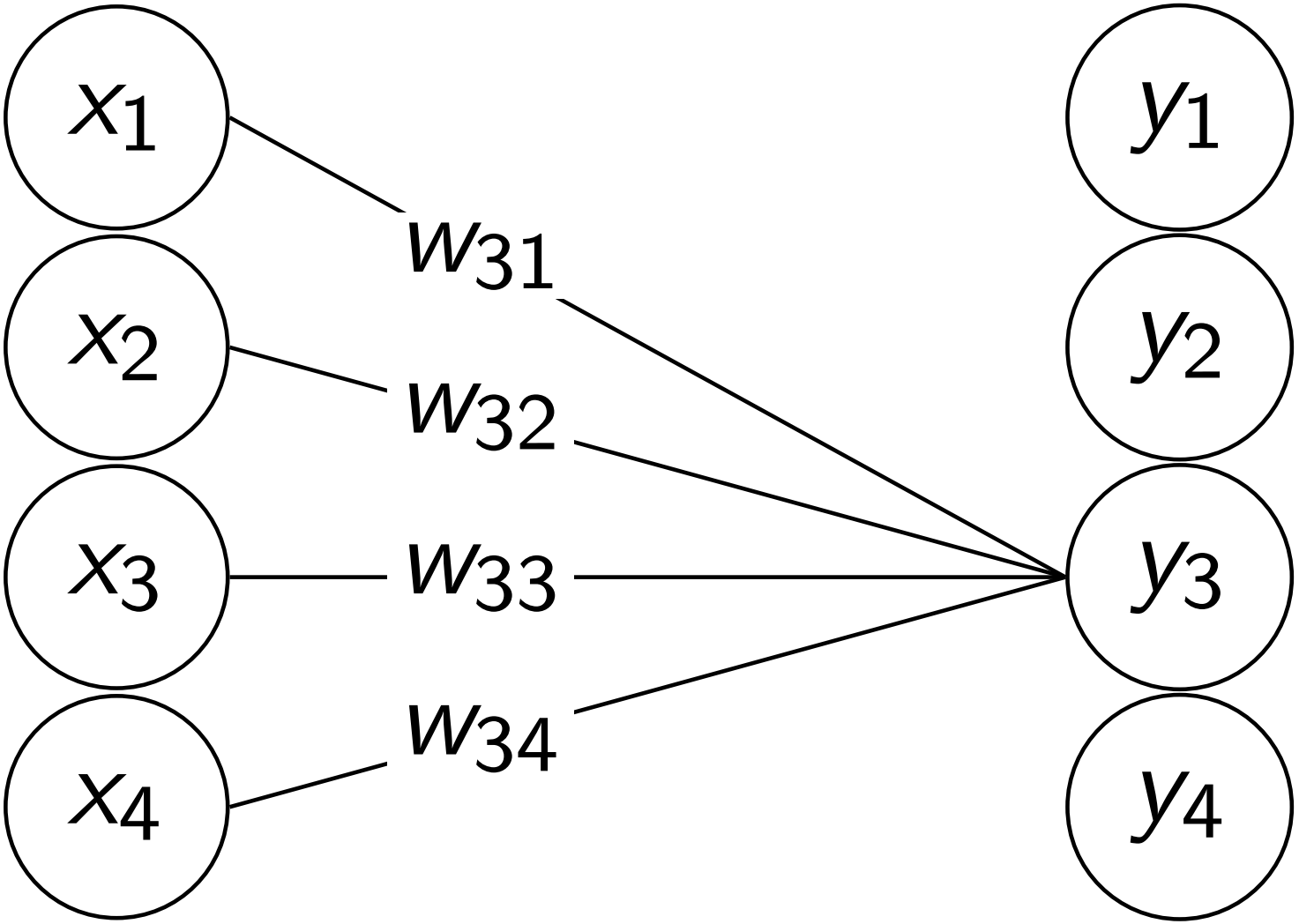
$$y_1 = x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14}$$

Basic Neural Net (MLP/FFN)



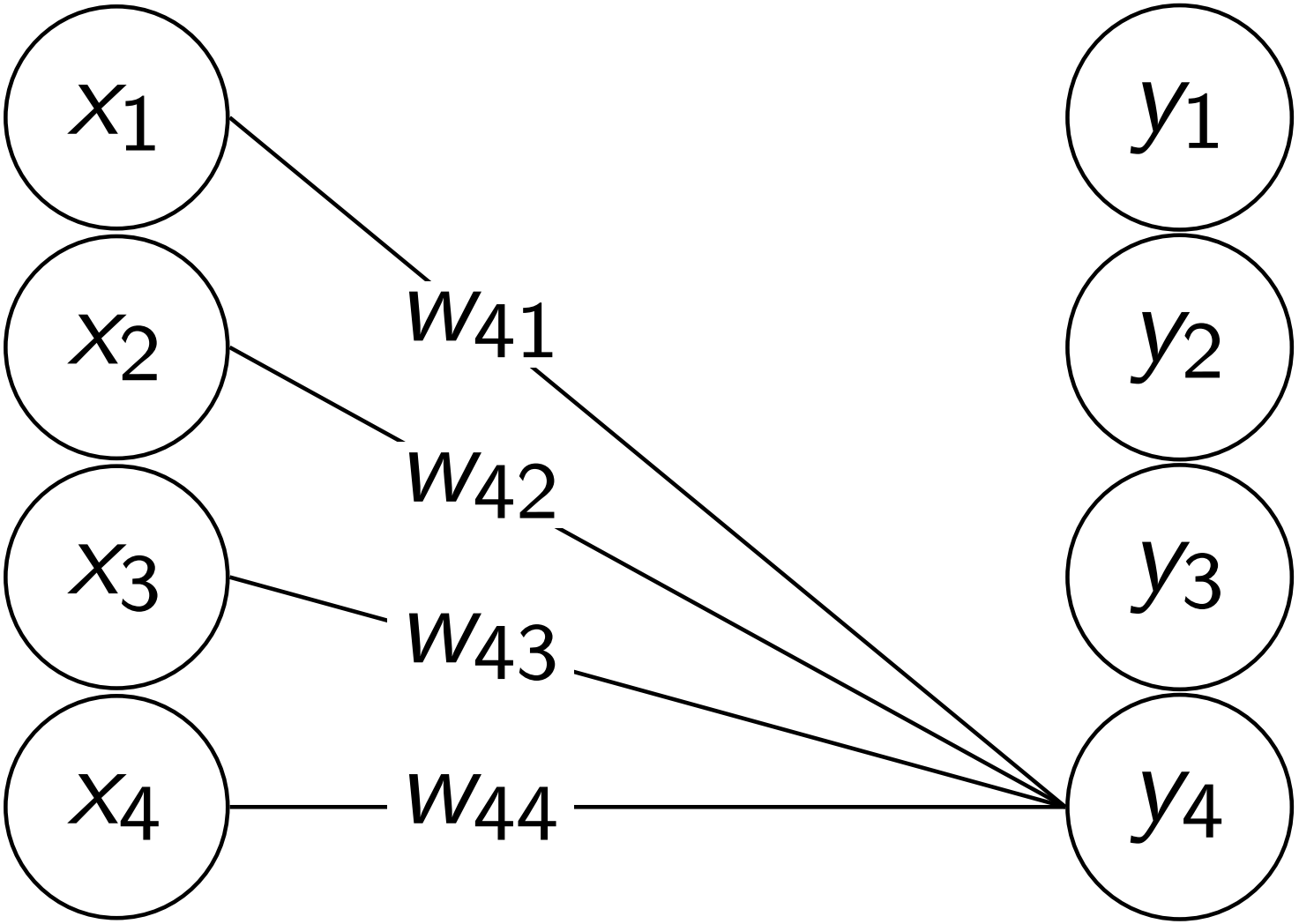
$$y_2 = x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24}$$

Basic Neural Net (MLP/FFN)



$$y_3 = x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34}$$

Basic Neural Net (MLP/FFN)



$$y_4 = x_1 w_{41} + x_2 w_{42} + x_3 w_{43} + x_4 w_{44}$$

Linear Algebra Interpretation

Here, notice that for each y_i, we are taking a weighted sum of the x elements. That is,

$$y_i = x_1 w_{i1} + x_2 w_{i2} + x_3 w_{i3} + x_4 w_{i4}.$$

Linear Algebra Interpretation

Here, notice that for each y_i, we are taking a weighted sum of the x elements. That is,

$$y_i = x_1 w_{i1} + x_2 w_{i2} + x_3 w_{i3} + x_4 w_{i4}.$$

Arranging the y_i into a vector, we get

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \\ w_{31} & w_{32} & w_{33} & w_{34} \\ w_{41} & w_{42} & w_{43} & w_{44} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}.$$

Linear Algebra Interpretation

We can generalize the number of x's and the number of y's.

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

Linear Algebra Interpretation

We can generalize the number of x's and the number of y's.

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

We say, n is the *input dimension* and m is the *output dimension*.

Computing with multiple x vectors.

Suppose we have multiple x vectors, for which we want to pass through our neural network. We can stack the multiple x vectors in the **batch**, which we compute with a matrix multiplication.

Computing with multiple x vectors.

Suppose we have multiple x vectors, for which we want to pass through our neural network. We can stack the multiple x vectors in the **batch**, which we compute with a matrix multiplication.

$$\begin{bmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(B)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ y_m^{(1)} & y_m^{(2)} & \cdots & y_m^{(B)} \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(B)} \\ x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ x_n^{(1)} & x_n^{(2)} & \cdots & x_n^{(B)} \end{bmatrix}.$$

Computing with multiple x vectors.

Suppose we have multiple x vectors, for which we want to pass through our neural network. We can stack the multiple x vectors in the **batch**, which we compute with a matrix multiplication.

$$\begin{bmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(B)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ y_m^{(1)} & y_m^{(2)} & \cdots & y_m^{(B)} \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(B)} \\ x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ x_n^{(1)} & x_n^{(2)} & \cdots & x_n^{(B)} \end{bmatrix}.$$

We write this succinctly as $Y = WX$, where $Y \in \mathbb{R}^{m \times B}$, $X \in \mathbb{R}^{n \times B}$, and $W \in \mathbb{R}^{m \times n}$.

Computing with multiple x vectors.

Suppose we have multiple x vectors, for which we want to pass through our neural network. We can stack the multiple x vectors in the **batch**, which we compute with a matrix multiplication.

$$\begin{bmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(B)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ y_m^{(1)} & y_m^{(2)} & \cdots & y_m^{(B)} \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(B)} \\ x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ x_n^{(1)} & x_n^{(2)} & \cdots & x_n^{(B)} \end{bmatrix}.$$

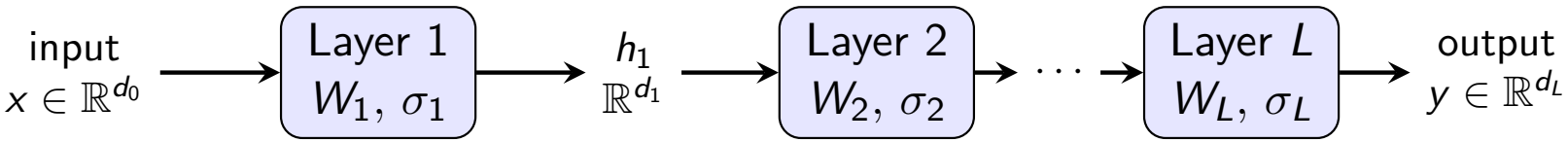
We write this succinctly as $Y = WX$, where $Y \in \mathbb{R}^{m \times B}$, $X \in \mathbb{R}^{n \times B}$, and $W \in \mathbb{R}^{m \times n}$.

New Representation

We have successfully mapped the batched computation of a **linear layer** a matrix multiplication!

Composing Layers

We can view W as a function that takes in an n-dimensional vector and maps it to an m-dimensional vector. $W : \mathbb{R}^n \to \mathbb{R}^m$. We create deep neural networks by *composing* these transformations, along with **nonlinearities** σ_i.



$$h_0 = x, \qquad h_\ell = \sigma_\ell(W_\ell h_{\ell-1}), \qquad y = h_L$$

Takeaways

1. A linear layer applied to a batch of inputs can be expressed as a single matrix multiplication, $Y = WX$.
2. Deep networks are composed of many of these linear transformations.
3. **Matrix multiplication therefore captures a large and reusable part of neural network computation.**

Why is this representation useful?

By optimizing a the MATMUL, we also optimize neural networks.